

Wat is een PWA?

Als web developer bouw ik websites in HTML, CSS en JavaScript. Wat ik lang niet wist: met dezelfde technieken kun je ook een volwaardige app maken die gebruikers kunnen installeren op hun telefoon of computer. Dat is precies wat een Progressive Web App is.

Volgens MDN Web Docs is een PWA een app die gebouwd is met webtechnologieën, maar die een gebruikerservaring biedt zoals die van een platformspecifieke app. Net als een website kan hij draaien op meerdere platforms vanuit één codebase, en net als een native app kan hij worden geïnstalleerd, offline werken en integreren met het apparaat.

Eén codebase, werkt overal. Dat is voor mij als developer het grote voordeel ten opzichte van een native app, waarbij ik voor iOS en Android apart moet bouwen.

Wat kan een PWA?

Meer dan ik verwachtte. Op whatpwacando.today staat een live overzicht van alle mogelijkheden, en de site is zelf ook een PWA, want zo zie ik meteen wat op mijn eigen apparaat wordt ondersteund.

Wat een PWA onder andere kan: installeren via een native dialoog, offline werken via een Service Worker, push-notificaties sturen ook als de app niet actief is, toegang tot de camera en microfoon, QR-codes en barcodes scannen, verbinding maken met Bluetooth- en NFC-apparaten, betalen via Apple Pay of Google Pay, bestanden openen via het bestandssysteem van het apparaat, en zelfs AR en VR ondersteunen.

Wel een kanttekening: niet elke browser ondersteunt alles. Chrome heeft de breedste ondersteuning, Safari op iOS is beperkter. Op whatpwacando.today zie ik per feature meteen of die werkt in mijn huidige browser.

Hoe bouw je een PWA?

Een PWA heeft technisch minimaal twee verplichte onderdelen.

Het eerste is het Web App Manifest. Dat is een JSON-bestand met metadata over de app: de naam, het icoon, de achtergrondkleur en de weergavemodus. Dit bestand vertelt de browser hoe de app eruit moet zien wanneer hij geïnstalleerd wordt, bijvoorbeeld of hij zonder adresbalk moet openen.

Het tweede is de Service Worker. Dat is een JavaScript-bestand dat op de achtergrond draait als proxy tussen de app en het netwerk. De Service Worker maakt offline functionaliteit mogelijk via caching, en verzorgt de communicatie tussen de achtergrondprocessen en de app zelf. Hij onderschept netwerkrequests, slaat resources op in de cache en serveert die wanneer er geen verbinding is.

Daarnaast moet een PWA worden geserveerd via HTTPS. Lokaal mag je ook via localhost werken.

PWABuilder: van website naar PWA

Wil ik een bestaande website omzetten naar een PWA, dan is pwabuilder.com een handig startpunt. PWABuilder is een open-source tool van Microsoft die het aanmaken van het manifest en de service worker automatiseert, en helpt de PWA te verpakken voor verschillende app stores.

Het werkt als volgt: ik vul mijn URL in, de tool analyseert mijn site en geeft een score op het manifest en de service worker. Vervolgens genereer ik de ontbrekende bestanden via de editor, download ik het pakket en upload ik de bestanden naar mijn server. Daarna kan ik de PWA publiceren in de Google Play Store, de Microsoft Store, de Meta Quest Store en de iOS App Store.

Wanneer kies ik voor een PWA?

Een PWA is voor mij de logische keuze als een project cross-platform moet werken, offline functionaliteit nodig heeft, of app-achtige features wil zonder twee native codebases te onderhouden. Voor zware graphics of diepe OS-integratie blijft native sterker, maar voor de meeste webprojecten is een PWA een serieuze optie die ik vaker zou overwegen dan ik tot nu toe deed.

Bronnen:

developer.mozilla.org

pwabuilder.com

<https://whatpwacando.today/>